

PCA and t-SNE

In this extended exercise you will explore the utility of PCA (and the SVD) and t-SNE for dimensionality reduction and, particularly, high-dimensional data visualization.

For the first few portions of the problem, you will work with some (real) RNA sequencing data. The experiment here (taken from the Gene Expression Omnibus, a public database of high-throughput sequencing data) studies the results of infecting human plasmacytoid dendritic cells (pDCs) with influenza A virus. pDCs are extremely important to viral defense and help direct immune responses by secreting proteins called interferons. (More information about this study is in Bajwa et al., “Cutting Edge: Critical Role of Glycolysis in Human Plasmacytoid Dendritic Cell Antiviral Responses,” *J Immunol* **195** (2016).)

The data you will analyze will consist of 10 samples (5 infected samples and 5 control samples) of (normalized) RNA sequence counts for 5,000 human genes.*

- (a) The first thing you need to do is construct the matrix X , which will put genes on the rows and samples on the columns. The data you need is in a CSV file on Canvas; the name of the file is `pH1N1-infection.txt`. The first five numbers in every row are for the infected samples, and the second five are for the control/uninfected samples. You will want to keep a record of which row of RNA counts goes with which gene (so, for example, a list of gene names in the same order as the rows).
- (b) You will start by doing a quality control analysis of the count data: does PCA in sample space separate the infected from the control samples? In order to do this, you need to do the appropriate SVD decomposition — don't write your own SVD solver, just use the one in `numpy.linalg`. Before you do the SVD, standardize your data appropriately. Remember, this is the operation that makes all the rows (or columns) of X have mean zero and standard deviation of one. Whether you standardize the rows or columns depends on whether you are doing the SVD/PCA in gene or sample space. (I have provided code for you to do the standardization.)
- (c) Compute the first two principal components — careful, in sample space these will be 10-dimensional vectors, since there are 10 samples. Plot the first principal component (on the x -axis) against the second principal component (on the y -axis), and plot the infected (first 5) points in a different symbol or color (or both) than the control (second 5) points. The scale on your plot will be nicer if you normalize the singular values by the appropriate size factor (as I discussed in class). Does it look like samples from the same group (infected/control) cluster together in the two dimensional space?
- (d) How much variance in the data is captured in PC1? What about PC2? What is the fraction of the total data variance captured by the first two PCs? Do the first two components capture the majority of the variance in the data?

While this may seem like a pretty simple analysis, it is often an essential step with sequencing experiments. If the experimental and control groups don't separate in PCA space, it's often not worth doing any further analysis, because you can't tell the treatment from the control. Next we will do PCA on the other dimension — the genes. When we perform PCA on the data matrix of

*The original set is around 23,000 genes. I'm providing you with just the 5,000 genes that differ the most between the experimental and control condition; the primary reason for doing this is to make the SVD calculation easier to do on a laptop.

genes $\times$ samples, the rows of the U matrix show the genome-wide expression of the *eigensamples*. Conversely, the rows of V^T contain the expression of each eigengene across the samples.

- (e) Prepare the data matrix on the gene (not sample) axis appropriately (as you did in (b)) above. Careful — you need to go back to the original X matrix. If you threw it away, recreate it from the data file.
- (f) Do the SVD appropriately and compute the principal components. You are going to want to use the option `full_matrices = False` when you call `numpy.linalg.svd`. Why is this? Recall from class that $X^T X$ and XX^T have the same singular values. This means there are only 10 nonzero singular values — so if you ask for 5000 of them, 4,990 of them will be zero. Hence, even for the gene dimension, only 10 PCs make sense. The `full_matrices = False` option throws out the stuff that is just going to be zero anyway and gives you a much more compact decomposition.
- (g) Make a bar plot of the genome wide expression of the first eigensample (first column of U). It will help if you sort the elements of this vector first (`numpy` has a function that will do this for you). This PC is the direction of primary change in the experimental data, and it is a linear combination of all the genes in the experiment. A large (close to ± 1) value for a particular gene means that gene participates a lot in the high-variance direction; a value near zero indicates that gene isn't really involved at all. Is there a clear separation into genes which are a big part of PC1 vs. ones that are not?
- (h) Despite the results of part (g), determine the names of the genes with the five largest positive scores in PC1 and those with the five largest negative scores in PC1. (Hint: Looking up the documentation for `numpy.argsort` is a very good idea.) Look up functions for these genes — the best place for this is the National Library of Medicine; select the “Gene” database from the search dropdown menu and then type the name of the gene in. You can also look up gene names on Wikipedia. Given the (very limited!) amount of information you have about the experiment, do any of the genes you found make sense?
- (i) Finally, we would like to see how a couple of the eigengenes vary across the samples. This information is contained in the rows of V^T . Make a plot of the first three rows of V^T . Do any of the eigengenes have patterns that are associated with the two experimental conditions?

The gene-space PC analysis may be a bit disappointing; this is not surprising given the makeup of the principal component itself, which doesn't have any clear separation between genes with large scores and those with small. There are other ways to look at these data, which we will do after we talk about clustering. For now, we are going to switch datasets and do a quick comparison of PCA and t-SNE. For the rest of this exercise, we will use a small portion of a very famous machine learning dataset; the MNIST database of handwritten digits.

- (j) The data you need is in `mnist-2000.pkl`. This is *not* a text file; it is a python binary created with the `pickle` module. The good thing about this is that `pickle` is very easy to use, and reading in the data is much easier than if I had stored it as a CSV file. Load the data using `data = pickle.load()`, which you should look up how to use. Be careful to open the file in `'rb'` mode (because you are trying to read a binary file). If you are successful, you should have a dictionary with two keys: `'images'` are the images (already flattened into vectors for you) and `'labels'` tell you which digit each element in the `images` list is a picture of.

- (k) Before doing any analysis, I encourage you — just for fun and as a sanity check — to actually look at a few digits. The best way to do this is to extract some image vector from the list in the data, and then use `imshow(numpy.reshape(img_vec,newshape=(28,28)))`. The function `imshow` is just a way to plot an image (a 2D array of grayscale or color values). The function `reshape` undoes the flattening I did and gives you a 2D image. If you have a hard time recognizing the digit (some of these are pretty challenging!) you can check the label that goes with it.

A comment about the reshaping of the images. PCA and t-SNE accept a data matrix in which each point is a vector of coordinates, so they cannot directly work on image data, which are matrices of pixels. In order to work with these images (for these and many other data science algorithms) we just convert the image into a vector, by concatenating all the rows (or columns) in the image matrix into one long vector. That's why your image vectors are size $784 = 28 \times 28$.

- (l) Plot PC1 vs PC2 scores of the 2,000 images, you will need to assemble the X matrix as done before, and don't forget to center! You are working in image space, not pixel space here — image space is like gene space in what you did above. Remember `full_matrices=False` here to save time and space. Color code the points in your according to image identity, so you can easily see how groups of the same digit are organized. When you make this plot, I'd recommend using `matplotlib.scatter` because you can give lists of x and y coordinates, along with a vector of colors that will color each point as you choose. Also, python supports a lot more colors than the ones they give simple strings for — hex colors are probably the easiest to use (Google "hex color picker" and you'll find some nice tools for color picking). How is PCA doing here? Which digits are clearly separated? Which ones are not?

You probably found PCA was plotting a lot of stuff on top of each other. Let's try t-SNE and see if moving away from a linear embedding helps better separate these digits in two dimensions.

- (m) As I mentioned in class, one issue with t-SNE is that it doesn't really dispense with PCA, in that if you have data with more than 50 variables/features/coordinates, you need to use PCA to reduce it to the suggested 50 before trying to get it down to 2. So return to your SVD of part (k) and make the 2000×50 dimensional data matrix consisting of the first 50 principal components as the columns.
- (n) Rejoice, for you do not have to write your own t-SNE code. Use the one in `scikit-learn` — you should look up the documentation. Their t-SNE function has a lot of parameters; we won't worry about most of them. The only ones we care about are `n_components` (you want to set that to 2) and the perplexity. For this calculation, set the perplexity equal to 50. Also, the way you call this is a little funky - there's a nice example of what you want to do in the `scikit-learn` documentation. Run it, and make a scatterplot of the result.[†] Does it separate the digits better?
- (o) Repeat part (m) for perplexity values of 25, 10, and 5. Remember, lower perplexity more severely limits what images are considered neighbors to a given image. Does the digit separation depend much on the perplexity? Which value do you like best? Which pair of digits look like they overlap the most? Does this make sense given their identity?

[†]Running t-SNE may take a bit of time. Be patient; just because it takes a couple of minutes does not mean anything is wrong.

PCA and t-SNE are powerful and ubiquitous tools for data visualization and dimensionality reduction. Perhaps you are already working in a lab that generates experimental data that would be interesting to use these techniques to investigate, or know of some data/images/etc. that would be worth looking at this way!